

Employee Management System

SERVICE STRUCTURE & PROJECT PLAN

Microservices-Based Application

1. API GATEWAY

Responsibilities

- Routing
- Authentication / JWT validation at the gateway level where configured
- CORS
- Centralized request entry point
- Gateway-level logging where required

Routes

Route	Target Service
/api/v1/employees/**	Employee Service
/api/v1/departments/**	Department Service
/api/v1/salaries/**	Salary Service
/api/v1/leaves/**	Leave Service
/api/v1/attendance/**	Attendance Service
/api/v1/support-tickets/**	Support Service
/api/v1/notifications/**	Notification Service

2. EMPLOYEE SERVICE

Owns Data

- Employee records

Responsibilities

- Add Employee
- View Employee
- View All Employees
- Search Employees
- Update Employee
- Delete Employee
- Employee-specific access control

Core Data

Field	Purpose
EmployeeId	Unique employee identifier
FirstName / LastName	Employee name

Field	Purpose
Email	Employee email; unique
Phone	Contact number
DateOfJoining	Joining date
DepartmentId	Reference to Department Service
Role	Employee role
EmploymentStatus	Current employment status
CreatedAt / UpdatedAt	Audit timestamps

APIs

- POST /api/v1/employees
- GET /api/v1/employees/{id}
- GET /api/v1/employees
- GET /api/v1/employees/search
- PUT /api/v1/employees/{id}
- DELETE /api/v1/employees/{id}

3. DEPARTMENT SERVICE

Owns Data

- Department records

Responsibilities

- Add Department
- View Department
- View All Departments
- Update Department
- Delete Department

Core Data

Field	Purpose
DepartmentId	Unique department identifier
DepartmentName	Unique department name
Description	Department description
CreatedAt / UpdatedAt	Audit timestamps

Business Rule

- A department cannot be deleted while employees are assigned to it.

APIs

- POST /api/v1/departments
- GET /api/v1/departments/{id}
- GET /api/v1/departments
- PUT /api/v1/departments/{id}

- DELETE /api/v1/departments/{id}

4. SALARY SERVICE

Owns Data

- Employee Salary
- Salary Components
- Salary Revisions
- Bonuses
- Overtime
- Payroll
- Payroll Items
- Payslips

Responsibilities

- Add / View / Update Employee Salary
- Add / Update / View Salary Components
- Create / View Salary Revision History
- Add / Update / View Bonus
- Submit / Approve / Reject / View Overtime
- Generate / View Payroll
- View Payroll Items
- Generate / View Payslips

Important Access Rule

- Employees can view permitted salary information and submit overtime, but cannot directly modify salary, bonus, or salary revision information.

APIs

- POST /api/v1/salaries
- GET /api/v1/salaries/{employeeId}
- PUT /api/v1/salaries/{employeeId}
- POST /api/v1/salaries/components
- PUT /api/v1/salaries/components/{id}
- POST /api/v1/salaries/revisions
- GET /api/v1/salaries/revisions/{employeeId}
- POST /api/v1/salaries/bonuses
- PUT /api/v1/salaries/bonuses/{id}
- POST /api/v1/salaries/overtime
- PUT /api/v1/salaries/overtime/{id}/approve
- PUT /api/v1/salaries/overtime/{id}/reject
- GET /api/v1/salaries/overtime/{employeeId}
- POST /api/v1/salaries/payroll

- GET /api/v1/salaries/payroll/{id}
- GET /api/v1/salaries/payroll/{id}/items
- POST /api/v1/salaries/payslips
- GET /api/v1/salaries/payslips/{id}

5. LEAVE SERVICE

Owns Data

- Leave records

Responsibilities

- Apply Leave
- View Leave
- View Leave History
- Approve Leave
- Reject Leave
- Track Leave Status

Core Data

Field	Purpose
LeaveId	Unique leave identifier
EmployeeId	Employee reference
LeaveType	Type of leave
StartDate / EndDate	Leave period
Reason	Reason for leave
Status	Pending / Approved / Rejected
AppliedAt	Application timestamp
ApprovedBy / ApprovedAt	Approval information
CreatedAt / UpdatedAt	Audit timestamps

Workflow

APIs

- POST /api/v1/leaves
- GET /api/v1/leaves/{id}
- GET /api/v1/leaves/employee/{employeeId}
- GET /api/v1/leaves/history/{employeeId}
- PUT /api/v1/leaves/{id}/approve
- PUT /api/v1/leaves/{id}/reject

6. ATTENDANCE SERVICE

Owns Data

- Attendance records

Responsibilities

- Check In
- Check Out
- View Attendance
- View Attendance History
- View Team Attendance

Core Data

Field	Purpose
AttendanceId	Unique attendance identifier
EmployeeId	Employee reference
AttendanceDate	Attendance date
CheckInTime	Check-in time
CheckOutTime	Check-out time
Status	Attendance status
CreatedAt / UpdatedAt	Audit timestamps

Business Rules

- One attendance record per employee per day.
- EmployeeId + AttendanceDate is unique.
- Check-out cannot be earlier than check-in.

APIs

- POST /api/v1/attendance/check-in
- PUT /api/v1/attendance/check-out
- GET /api/v1/attendance/{id}
- GET /api/v1/attendance/employee/{employeeId}
- GET /api/v1/attendance/team

7. SUPPORT SERVICE

Owns Data

- Support tickets

Responsibilities

- Create Ticket
- View Ticket
- View Tickets
- Assign Ticket
- Update Ticket Status
- Resolve Ticket
- Close Ticket

Core Data

Field	Purpose
TicketId	Unique ticket identifier
EmployeeId	Ticket creator / employee reference
Title	Ticket title; required
Description	Ticket description; required
Priority	Ticket priority
Status	Ticket workflow status
AssignedTo	Assigned support user
CreatedAt / UpdatedAt	Audit timestamps
ResolvedAt	Resolution timestamp

Workflow

APIs

- POST /api/v1/support-tickets
- GET /api/v1/support-tickets/{id}
- GET /api/v1/support-tickets
- PUT /api/v1/support-tickets/{id}/assign
- PUT /api/v1/support-tickets/{id}/status
- PUT /api/v1/support-tickets/{id}/resolve
- PUT /api/v1/support-tickets/{id}/close

8. NOTIFICATION SERVICE

Owns Data

- Notification records

Responsibilities

- Receive supported Kafka events
- Create notifications
- View notifications
- Mark notification as read

Core Data

Field	Purpose
NotificationId	Unique notification identifier
EmployeeId	Recipient employee reference
Title	Notification title
Message	Notification content
Type	Notification event/type
IsRead	Read status
CreatedAt	Creation timestamp

Supported Events

- Leave Approved
- Leave Rejected
- Salary Updated
- Support Ticket Updated
- Overtime Approved

APIs

- GET /api/v1/notifications
- PUT /api/v1/notifications/{id}/read

9. SUPPORTING INFRASTRUCTURE

Component	Purpose
Eureka Service Discovery	Service registration and discovery.
API Gateway	Single client-facing entry point and routing.
Apache Kafka	Asynchronous event-driven communication.
PostgreSQL	Persistent storage; database owned by each service.
Security	Authentication, JWT, RBAC and resource-level authorization.
API Versioning	Versioned public API contracts.
Central Logging	Centralized service/application logs.
Swagger / OpenAPI	API documentation.
Bruno	API testing.
xUnit	Unit testing.

10. PROJECT FOLDER STRUCTURE

Following the organization used in the previous CodeCanvas project, but adapted to .NET:

```
Employee-Management-System/ ■■■■ backend/ ■■■■ api-gateway/ ■■■■ employee-service/ ■■■■
■■■■ department-service/ ■■■■ salary-service/ ■■■■ leave-service/ ■■■■
attendance-service/ ■■■■ support-service/ ■■■■ notification-service/ ■■■■ frontend/ ■■■■
database/ ■■■■ docs/ ■■■■ diagrams/ ■■■■ tests/ ■■■■ README.md
```

11. STANDARD .NET SERVICE STRUCTURE

```
employee-service/ ■■■■ Controllers/ ■■■■ Services/ ■■■■ Interfaces/ ■■■■
Implementations/ ■■■■ Repositories/ ■■■■ Interfaces/ ■■■■ Implementations/ ■■■■ Models/
■■■■ DTOs/ ■■■■ Requests/ ■■■■ Responses/ ■■■■ Data/ ■■■■ Exceptions/ ■■■■ Middleware/ ■■■■
Configuration/ ■■■■ Program.cs ■■■■ appsettings.json
```

Folder Ownership Rule

If you are writing...	Folder
REST API	Controllers
Business logic	Services / Implementations
Method definitions	Services / Interfaces
Database access	Repositories
Database entities / domain models	Models
Request / response models	DTOs
DbContext and database configuration	Data

If you are writing...	Folder
Exception handling	Exceptions / Middleware
Application configuration	Configuration
Application startup	Program.cs

12. SERVICE COMMUNICATION

13. DEVELOPMENT ORDER

Phase	Work
1	Solution and repository structure
2	Service projects and basic configuration
3	Employee Service
4	Department Service
5	Salary Service
6	Leave Service
7	Attendance Service
8	Support Service
9	Notification Service
10	API Gateway and service discovery
11	Kafka event communication
12	Security and API versioning
13	Central logging and Swagger
14	Bruno, xUnit and integration testing
15	Documentation and final integration

14. IMPORTANT PROJECT RULE